Threshold-Aware Similarity Caching for Named Data Networks: Bounding Retry Delay under Congestion using ns-3

October 8, 2026

S-2-T-7: Team Members

1. 241CS257, Somyak Priyadarshi Mohanta, [redacted]
2. 241CS243, Pranav Shaji, [redacted]
3. 241CS222, Chris Tony, [redacted]

Abstract

Background

Caching is one of the oldest ways to make content delivery faster and lighter on the network. Whether it is a CDN, a cache at the mobile edge, or the in-network caches of Named Data Networking (NDN), the rule has always been the same: a cache can serve a request only if it holds the exact object the user asked for [1]. Similarity caching loosens that rule. If the exact object is missing, the cache hands back the closest thing it has. For a lot of applications this is good enough, and image retrieval, recommendation systems and semantic caching of LLM queries are typical examples [2], [3]. The catch is that the user decides whether the answer is close enough. If it is not, the request simply goes out again.

State-of-the-Art

The work closest to ours is by Nakamura and Kamiyama [4], who studied similarity caching over a general network of caches. Their main contribution is the similar-content delivery delay, a metric seen from the user's side that also counts the time lost to repeated requests. To keep the analysis tractable, they assume that each cache answers with a fixed probability α and that links have unlimited bandwidth, with no queuing and no loss. All of their results come from numerical evaluation on a tandem topology of five caches. Earlier work looked at a single similarity cache, or at how to place content across a network of them [2], [5]. As far as we can tell, nobody has yet looked at how requests and retries actually behave at the packet level.

Issues and Challenges

Moving similarity caching from an analytical model into a real multi-hop network raises a few problems:

- **Correlated Retries**: The model treats each retry as a fresh, independent draw of what the caches hold. Real caches do not behave like that. Their contents hardly change between one retry and the next, so a user who rejects an answer is likely to get the very same answer again. When we re-ran the model on the same topology but kept the cache contents fixed across retries, the mean delay turned out to be underestimated by roughly a factor of two at $\alpha = 0.5$, and by more than sixty times at $\alpha = 0.9$.
- **Blind Caches**: A cache has no idea how much similarity the user is willing to accept, so it may send back an object that is bound to be rejected.
- **NDN Name Matching**: In NDN, a Data packet is delivered only if its name matches the Interest. Returning a different (even if similar) object therefore means changing both the packet format and the forwarder.
- **Congestion and Loss**: Whether it pays to answer from a nearby cache depends on how congested and lossy the upstream path is, and the current model has no way to capture this.

Objectives

Our goal is to design threshold-aware similarity caching for NDN, implement it, and evaluate it in ns-3 using the ndnSIM module [6]. Concretely, we plan to:

1. Build a similarity-based Content Store, along with consumer and producer applications, in ndnSIM, and use it to reproduce the base paper's results on the same tandem topology.

2. Measure how much retry delay blind similarity caching really causes when cache contents persist between requests, and compare this with the analytical model.

3. Extend the Interest so that it carries the lowest similarity the user will accept and a cap on the number of retries. A cache then answers only if its best match clears that threshold.

4. Design a forwarding strategy that decides whether to answer locally or forward upstream based on the measured upstream delay, and test it under link congestion and burst packet loss.

5. Compare our scheme with the base scheme on mean and 95th percentile delivery delay, delivered similarity, upstream traffic and fairness across clients.

In short, we want to find out whether two small changes, a richer request format and a smarter decision rule at each cache, are enough to make similarity caching reliable in multi-hop networks. In such networks a bad answer from a nearby cache is not free, because it costs the user an entire retry.

Methodology Overview

The system we propose is built from the following pieces:

- ns-3 with the ndnSIM module as the simulation platform
- A similarity-based Content Store at every cache node, in place of exact name lookup
- Consumer applications that check the similarity and retry when needed, and a producer that acts as the origin server
- A custom forwarding strategy that makes the answer-or-forward decision
- Point-to-point links where bandwidth, delay, queue size and burst loss can all be configured
- Python and MATLAB scripts to compare the simulation output with the analytical model

References

[1] L. Zhang et al., "Named Data Networking," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 66-73, 2014.

[2] G. Neglia, M. Garetto, and E. Leonardi, "Similarity Caching: Theory and Algorithms," in *IEEE/ACM Transactions on Networking*, vol. 30, no. 2, pp. 475-486, 2022.

[3] L. G. Schroeder et al., "vCache: Verified Semantic Prompt Caching," arXiv:2502.03771, 2025.

[4] R. Nakamura and N. Kamiyama, "Analysis of Similarity Caching on General Cache Networks," in *IEEE Access*, vol. 12, pp. 163338-163348, 2024.

[5] M. Garetto, E. Leonardi, and G. Neglia, "Content Placement in Networks of Similarity Caches," *Computer Networks*, vol. 201, Art. no. 108570, 2021.

[6] S. Mastorakis, A. Afanasyev, and L. Zhang, "On the Evolution of ndnSIM: an Open-Source Simulator for NDN Experimentation," *ACM SIGCOMM Computer Communication Review*, vol. 47, no. 3, pp. 19-33, 2017.